

Physics as Code: From Scans to Theorems with ITP APIs



Joseph Tooby-Smith

Department of Computer Science, University of Bath

a project with Sven Krippendorf
DAMTP and Cavendish Laboratory, University of Cambridge

A warning

This talk is **deliberately provocative.**

Please indulge me for the next hour.

PART 1 OF 3

Setting the scene

the problem



1 **Setting the scene** • 2 Interactive theorem provers • 3 Our explicit example



My background

I am a theoretical physicist by training.

For a long time I have been a **consumer** of computer algebra systems, not producing the code.

Examples of my own work

Flatland: abelian extensions of the Standard Model with semi-simple completions

Joe Davighi (Zurich U.), Joseph Tooby-Smith (Cornell U., LEPP)

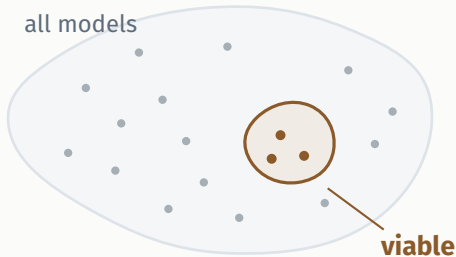
June 2022

Superflocinaucinihilipilification: semisimple unifications of any gauge theory

Andrew Gomes, Maximilian Ruhdorfer, Joseph Tooby-Smith

June 2023

Both fall into a characteristic class of example problem



exclusion plots • symmetry constraints • the swampland

Combinatorial explosion

Typical way these problems are approached

Use a computer algebra system (in practice, **Mathematica**) to either:

- 1 run a brute-force scan**
- 2 use a series of constructions to lift to the right answer**

Typical way these problems are approached

Use a computer algebra system (in practice, **Mathematica**) to either:

- 1 run a brute-force scan**
- 2 use a series of constructions to lift to the right answer**

I have written these scans. The code was long, unchecked, and used exactly once.

A controversial claim

**It would be better to use an
interactive theorem prover
than a computer algebra system
to solve these sorts of problems.**

PART 2 OF 3

Interactive theorem provers

the tool



1 Setting the scene • **2 Interactive theorem provers** • 3 Our explicit example



The plan

- 1 What is an interactive theorem prover?**
- 2 The defence of the claim.**
- 3 The case against the claim.**
- 4 The compromise.**

What is an interactive theorem prover?

*A programming language in which you can write mathematical theorems proofs, and definitions, with mathematical **correctness** and **consistency** guaranteed. And the definitions you write are **executable**.*

The four main interactive theorem provers

Rocq

Isabelle/HOL

Lean

Agda

This talk is in **Lean**.

A rapid overview of Lean

Built on type theory

Lean knows the **type** of everything, and checks that things have the type expected.

$$\underbrace{p}_{\text{proof}} : \underbrace{P}_{\text{theorem}} \quad \underbrace{7}_{\text{term}} : \underbrace{\mathbb{N}}_{\text{type}}$$

A definition, a theorem, a proof

```
def energy (m v : Real) : Real :=  
  m * v ^ 2 / 2  
  
theorem energy_nonneg  
  (m v : Real) (hm : 0 <= m) :  
    0 <= energy m v := by  
  unfold energy  
  positivity
```

The main libraries



leanprover-community / mathlib4

The math library of Lean 4.

public

Lean 4



Lean



3,989 stars



1,635 forks



776 contributors



started 2021

The main libraries



leanprover-community / physlib

A community-driven project to digitalise results from physics into Lean.

public

Lean 4

● Lean ★ 717 stars 🍴 176 forks 👤 106 contributors ⌚ started 2024

The main libraries



leanprover / cslib

The computer science library of Lean.

public

Lean 4



Lean



683 stars



190 forks



48 contributors



started 2025

Lean, live



Follow along example

live.lean-lang.org 

Relation to AI and maths in the news

MAY 2026

The unit-distance problem

Erdős, 1946.

Disproved.

OpenAI, internal model

JUL 2026

The Jacobian conjecture

Keller, 1939.

Counterexample in $\mathbb{C}^3$.

Alpoge, with Claude Fable 5

AUG 2026

Ten open problems

Each with a Lean 4 certificate.

OpenAI “Astra”

AUG 2026

Zeros on the critical line

41.6% $\rightarrow$ 67.2%.

Not a proof of RH.

Anthropic, checked in Lean



THE DEFENCE

“It would be better to use an interactive theorem prover than a computer algebra system to solve these sorts of problems.”

Here is why.

The defence, in five points

1 Interactive theorem provers give you **trust**.

every step is machine-checked, so a bug cannot hide

The defence, in five points

1 Interactive theorem provers give you **trust**.

every step is machine-checked, so a bug cannot hide

2 Definitions, lemmas and executable functions in one language: the **full reasoning** is in the code.

and a slow definition can be swapped for a fast algorithm, with a proof they agree

The defence, in five points

1 Interactive theorem provers give you **trust**.

every step is machine-checked, so a bug cannot hide

2 Definitions, lemmas and executable functions in one language: the **full reasoning** is in the code.

and a slow definition can be swapped for a fast algorithm, with a proof they agree

3 They are **AI-safe**.

a feedback loop: AI generates, Lean verifies

The defence, in five points

1 Interactive theorem provers give you **trust**.

every step is machine-checked, so a bug cannot hide

2 Definitions, lemmas and executable functions in one language: the **full reasoning** is in the code.

and a slow definition can be swapped for a fast algorithm, with a proof they agree

3 They are **AI-safe**.

a feedback loop: AI generates, Lean verifies

4 Lean is set up in **monolithic libraries**.

one coherent, composable whole in which everything works with everything

The defence, in five points

1 Interactive theorem provers give you **trust**.

every step is machine-checked, so a bug cannot hide

2 Definitions, lemmas and executable functions in one language: the **full reasoning** is in the code.

and a slow definition can be swapped for a fast algorithm, with a proof they agree

3 They are **AI-safe**.

a feedback loop: AI generates, Lean verifies

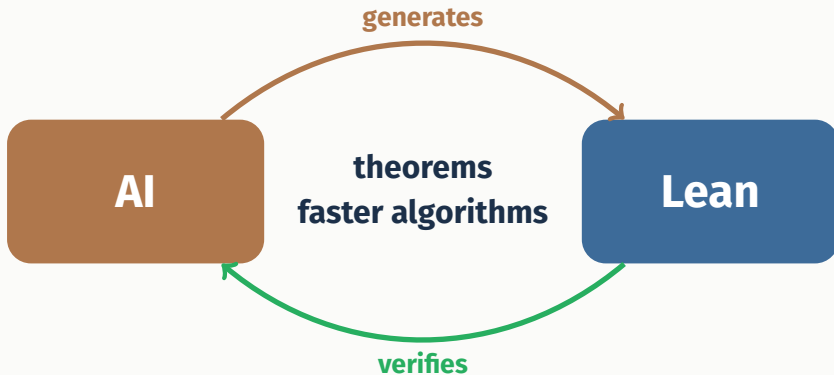
4 Lean is set up in **monolithic libraries**.

one coherent, composable whole in which everything works with everything

5 Theorems can **tame the combinatorial explosion**.

intuition becomes theorems that construct the answer, where a brute-force scan is infeasible

AI and Lean





THE CASE AGAINST

And here is why not.

The case against, in three points

1 Decades of work have gone into **computer algebra systems**.

Mathematica, Maple, FORM, and the many other tools built by this community

The case against, in three points

- 1 **Decades of work have gone into computer algebra systems.**

Mathematica, Maple, FORM, and the many other tools built by this community

- 2 **So many things a physicist wants to do are easy in a CAS.**

and, today, not available inside an ITP

The case against, in three points

- 1 **Decades of work have gone into computer algebra systems.**

Mathematica, Maple, FORM, and the many other tools built by this community

- 2 **So many things a physicist wants to do are easy in a CAS.**

and, today, not available inside an ITP

- 3 **Adapting these algorithms to an ITP is a huge amount of work.**

verification is part of the story: the code must come with its proof



THE COMPROMISE

**Can we get the best
of both worlds?**

We can.

Tactics that do the algebra for you, **verified**:

ring grind

...and the beginnings of a verified computer algebra library (early days):



leanprover / hex

Verified computational algebra in Lean 4: aggregator for the released hex libraries.

public

Lean 4

● Lean

★ 21 stars

🔗 2 forks

👥 2 contributors

🕒 started 2026

Bring the computer algebra into the theorem prover.

(This is my call to action for this community.)

In particular, I would love to see, in **Physlib**:

**tactics and CAS tools that make
the physicist's life easier.**

PART 3 OF 3

Our explicit example

the proof



1 Setting the scene • 2 Interactive theorem provers • **3 Our explicit example**





THE EVIDENCE

**The defence made five promises.
Here they are, in practice.**

The problem, mathematically

$$(q_{H_d}, q_{H_u}, Q_5, Q_{10})$$

$q_{H_d}, q_{H_u} : \text{Option } \mathbb{Z}$

$Q_5, Q_{10} : \text{Finset } \mathbb{Z}$

all drawn from a finite set

Large. Finite.



Find v .

ONE CONSTRAINT – FORBID THE DANGEROUS COUPLING $\bar{5} \bar{5} 10$

$$\forall a, b \in Q_5, c \in Q_{10} : \quad a + b + c \neq 0$$

The physics: $SU(5)$ model building

AMBIENT SPACE $\mathcal{A}$

SUSY $SU(5)$ + Abelian symmetry

Higgs: down-type, up-type q_{H_d}, q_{H_u}

Matter: $\bar{5}, 10$ Q_5, Q_{10}

charges from finite sets, from F-theory

VIABLE $\mathcal{V}$

Top Yukawa allowed

Dangerous operators forbidden

No regeneration

all conditions on $(q_{H_d}, q_{H_u}, Q_5, Q_{10})$

Lives in Physlib

 `leanprover-community/physlib`

`Physlib/Particles/SuperSymmetry/SU5`

`Physlib/StringTheory/FTheory`

12,171 lines of Lean.

Build an API

```
SU5/  
FieldLabels.lean  
Potential.lean  
ChargeSpectrum/  
  Basic.lean  
  OfFieldLabel.lean  
  OfPotentialTerm.lean  
  AllowsTerm.lean  
  MinimallyAllowsTerm/  
    Basic.lean  
    FinsetTerms.lean  
    OfFinset.lean  
MinimalSuperSet.lean  
Completions.lean  
PhenoConstrained.lean  
PhenoClosed.lean  
Yukawa.lean  
Map.lean  
ZMod.lean
```

Small pieces

Definitions, lemmas, theorems
around one data structure
composing into the result

Charge spectra as the basic object

```
structure ChargeSpectrum ( $\mathcal{Z}$  : Type :=  $\mathbb{Z}$ ) where  
  qHd : Option  $\mathcal{Z}$   
  qHu : Option  $\mathcal{Z}$   
  Q5 : Finset  $\mathcal{Z}$   
  Q10 : Finset  $\mathcal{Z}$ 
```

One data structure. Everything else hangs off it.

Restrict q_{H_d} , q_{H_u} , Q_5 and Q_{10} to draw from finite subsets of $\mathcal{Z}$, and the type becomes **our ambient space** $\mathcal{A}$.

Instances

```
instance emptyInst : EmptyCollection (ChargeSpectrum  $\mathcal{Z}$ ) where
  emptyCollection := ⟨none, none, {}, {}⟩

instance hasSubset : HasSubset (ChargeSpectrum  $\mathcal{Z}$ ) where
  Subset x y :=
    x.qHd.toFinset  $\subseteq$  y.qHd.toFinset  $\wedge$ 
    x.qHu.toFinset  $\subseteq$  y.qHu.toFinset  $\wedge$ 
    x.Q5  $\subseteq$  y.Q5  $\wedge$ 
    x.Q10  $\subseteq$  y.Q10
```

The whole argument is stated using $\subseteq$.

Once it is an instance, **Mathlib's lemmas about $\subseteq$ apply.**

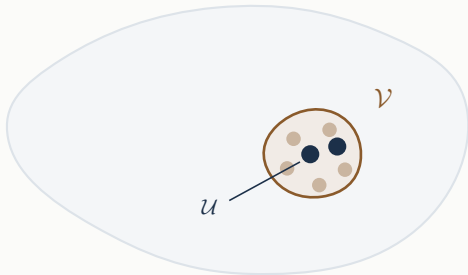
The verbs of the API

The things you want to ask about, and **the questions you want to ask of them.**

```
def AllowsTerm (x : ChargeSpectrum  $\mathcal{Z}$ ) (T : PotentialTerm) : Prop :=  
  0  $\in$  ofPotentialTerm x T  
  
def IsPhenoConstrained (x : ChargeSpectrum  $\mathcal{Z}$ ) : Prop :=  
  x.AllowsTerm  $\mu$   $\vee$  x.AllowsTerm  $\beta$   $\vee$  x.AllowsTerm  $\Lambda$   $\vee$  x.AllowsTerm W2  $\vee$   
  x.AllowsTerm W4  $\vee$  x.AllowsTerm K1  $\vee$  x.AllowsTerm K2  $\vee$  x.AllowsTerm W1
```

$\neg$ AllowsTerm x Λ is the $\forall a, b \in Q_5, c \in Q_{10} : a + b + c \neq 0$ from earlier.
 $\neg$ IsPhenoConstrained bundles it and seven more.

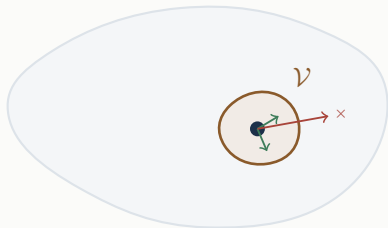
Step 1 (theorem): a small set of seeds



Every element of $\mathcal{V}$ contains one of these as a subset.

```
lemma allowsTerm_iff_subset_minimallyAllowsTerm :  
  x.AllowsTerm T  $\leftrightarrow$   
   $\exists$  y  $\in$  powerset x, y.MinimallyAllowsTerm T
```

Step 2 (theorem): which additions are allowed



Add a charge to Q_5 or Q_{10} , or fill in q_{H_d} or q_{H_u} .

Some additions stay inside $\mathcal{V}$.

Some leave it.

**And once you leave,
you cannot get back in.**

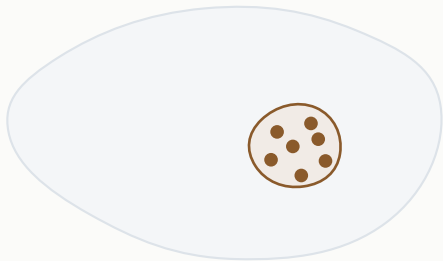
```
lemma isPhenoConstrained_mono
  {x y : ChargeSpectrum  $\mathcal{Z}$ } (h : x  $\subseteq$  y)
  (hx : x.IsPhenoConstrained) :
  y.IsPhenoConstrained
```

Step 3 (theorem): a classification

Steps 1 and 2 give **easy-to-check** conditions on $\mathcal{V}$:

$$x \in \mathcal{V} \iff \text{viability conditions on } x$$

Step 4 (compute): $\mathcal{V}_0$



Run the construction with ordinary techniques.

Out comes a list, $\mathcal{V}_0$.

`completeMinSubset` `minimalSuperSet` `viableCharges`

Step 5 (theorem): $\mathcal{V}_0 = \mathcal{V}$

STEP 3

$x \in \mathcal{V} \iff$ viability conditions

STEP 4

$\mathcal{V}_0$, a list you can print



```
lemma mem_viableCharges_iff {I} {x : ChargeSpectrum}
  (hsub : x ∈ ofFinset I.allowedBarFiveCharges I.allowedTenCharges) :
  x ∈ viableCharges I  $\iff$ 
    AllowsTerm x topYukawa  $\wedge$ 
     $\neg$  IsPhenoConstrained x  $\wedge$ 
     $\neg$  YukawaGeneratesDangerousAtLevel x 1  $\wedge$ 
    IsComplete x := ...
```

The claims made good

1 **Trust:** the final theorem is certified.

$\mathcal{V}_0 = \mathcal{V}$, every step checked, so a bug cannot hide

The claims made good

1 **Trust:** the final theorem is certified.

$\mathcal{V}_0 = \mathcal{V}$, every step checked, so a bug cannot hide

2 **Full reasoning:** the argument lives in the code.

the lemmas you just saw are the physicist's intuition, made precise

The claims made good

1 **Trust:** the final theorem is certified.

$\mathcal{V}_0 = \mathcal{V}$, every step checked, so a bug cannot hide

2 **Full reasoning:** the argument lives in the code.

the lemmas you just saw are the physicist's intuition, made precise

3 **AI-safe:** AI could have written this code.

with Lean checking every line

The claims made good

1 **Trust:** the final theorem is certified.

$\mathcal{V}_0 = \mathcal{V}$, every step checked, so a bug cannot hide

2 **Full reasoning:** the argument lives in the code.

the lemmas you just saw are the physicist's intuition, made precise

3 **AI-safe:** AI could have written this code.

with Lean checking every line

4 **Monolithic libraries:** the API now lives in Physlib.

12,171 lines, ready for the next project to build on

The claims made good

1 **Trust:** the final theorem is certified.

$\mathcal{V}_0 = \mathcal{V}$, every step checked, so a bug cannot hide

2 **Full reasoning:** the argument lives in the code.

the lemmas you just saw are the physicist's intuition, made precise

3 **AI-safe:** AI could have written this code.

with Lean checking every line

4 **Monolithic libraries:** the API now lives in Physlib.

12,171 lines, ready for the next project to build on

5 **Combinatorial explosion:** we never had to scan.

$\mathcal{V}_0$ was built up from minimal seeds, then certified

In conclusion

I have made a **controversial claim**. You do not need to believe it.

But I hope you take away some of the **benefits** of interactive theorem provers, and some of the things you can **do** with them.

Thank you for watching



Joseph Tooby-Smith

 `leanprover-community/physlib`

 `josephitoobysmith.com`